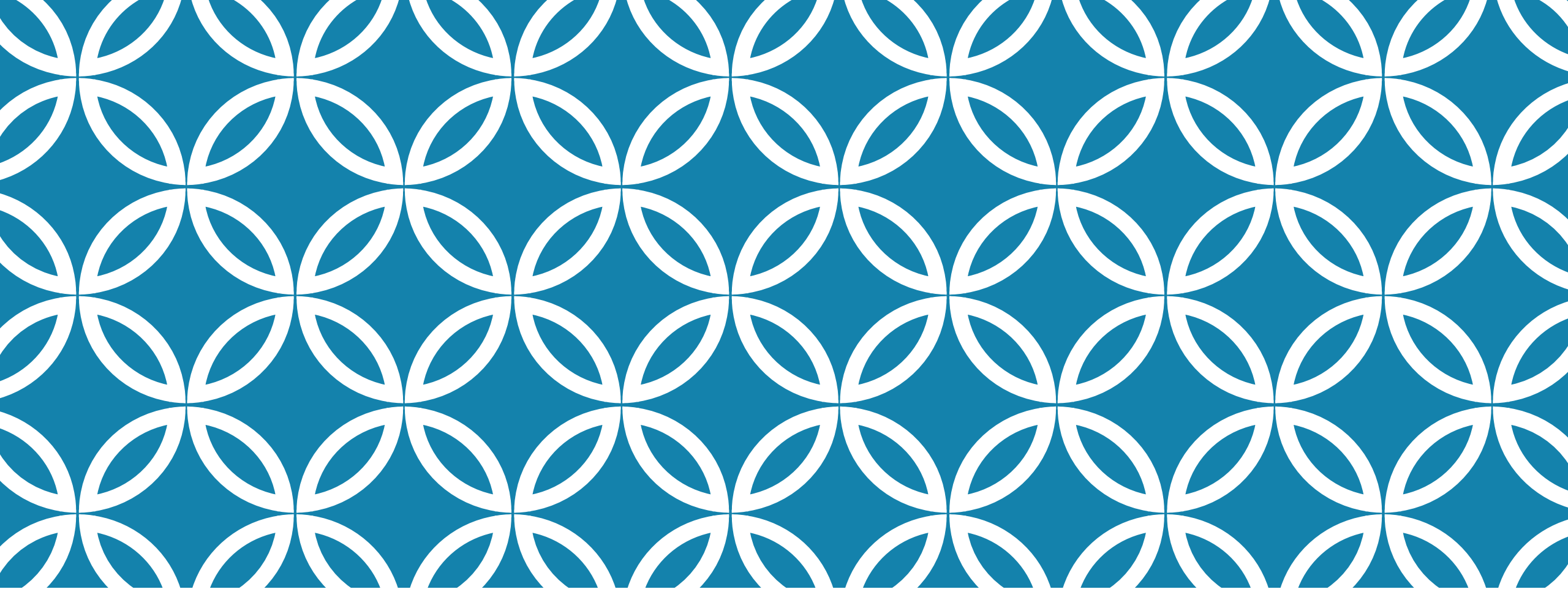




CSE455 - SOFTWARE TESTING

Chapter 6



OUTLINE

Test Organization

Testing Strategies

Test Planning, Control and Monitoring

Defect Management

Configuration Management

Relevant Standards and Norms

Conclusion

INDEPENDENT TESTING

Testing is needed throughout a software product's life

To ensure quality, testing should be well-coordinated with development

It is more beneficial and efficient to have separate development and testing teams organized independently

INDEPENDENT TESTING CONT..

- + Unbiased perspective – Testers see things differently from developers
- + More defects found – Independent view helps catch missed errors
- + Critical thinking – Testers challenge assumptions made in code or specifications
- Poor communication – If teams are too separate, it can hurt collaboration
- Limited knowledge – Testers may not fully understand the product
- Reduced developer responsibility – Developers may rely too much on testers

MODELS OF INDEPENDENT TESTING

Developer Testing

Independent Testing

Independent Testing Teams

Test Specialists

External Service Providers

MODELS OF INDEPENDENT TESTING

Developer Testing

- Developers test each other's code (not their own)
- Use techniques like buddy or pair testing

Independent Testing

- Testers are part of the team but only focus on testing
- Team members who are assigned for this role perform all testing activities within the team

MODELS OF INDEPENDENT TESTING

Independent Testing Teams

- Separate teams led by test leaders (test manager)
- May include experts from other departments when needed

Test Specialists

- Expert in specific aspect of testing (e.g. security, reliability)
- Support project with coaching, automation and training

External Service Providers

- Outside companies handle some or all testing tasks
- Can be insourced (in-house) or outsourced

WHEN TO USE EACH TESTING MODEL

Component Testing

Developers – model 1

Dedicated testers if available – model 2

Integration Testing

If components to be integrated are developed by same team – model 1 or 2

If different teams – model 3, 4, 5

System Testing

Need complete independence from developers – model 3, 4, 5

WHEN TO USE EACH TESTING MODEL

Based on Development model

- **V-model projects**
 - Ensure strict separation of development and test – model 3-5
- **Agile projects**
 - Teams work closely – model 1, 2
 - But model 3-5 can also benefit
 - Specialists support automation and CI/CD
 - Model 2, 5 can manage automated tests efficiently
 - Model 3, 5 scale well for large teams or projects
 - Model 4, 5 provide test coaching

TESTING ROLES

Efficient, professional testing needs staff with experience in testing, and technical fields

Main roles involved

- Test Manager
- Tester – often split into test analyst, test administrator,
- Multidisciplinary IT Specialists
- External Testing Service Providers

TEST MANAGER

Responsibilities

- Plan and lead all testing activities
- Draft test strategies and policies
- Build, support, and coach test teams
- Align with project/product manager
- Plan resources (budget, staff, tools, environment)

TEST MANAGER CONT..

Project Level Contributions

- Create test plans, schedules, and status reports
- Oversee test case design, automation and execution
- Ensure defect/configuration management
- Define and evaluate test metrics

Qualifications

- Background in software testing, QA, project and HR management
- Formal test management training

TESTER

A person performing or supporting testing activities (could be developer, business analyst, domain expert etc.)

Tasks

- Review test plans and cases
- Prepare test data
- Run manual and *exploratory* tests
- Use testing tools and run automated tests
- Log results and report defects

TESTER CONT..

Skills needed

- Basic IT knowledge
- Domain/application expertise
- Familiarity with test tools
- Certified Tester qualification

Soft Skills

- Teamwork, diplomacy, assertiveness
- Creativity, precision, quick learner
- Critical thinking and problem solving

MULTIDISCIPLINARY TEAMS

When testing requires specific technical or domain expertise

Common specialists are database designers, network engineers, industry/domain experts

Managing diverse team is complex and requires strong leadership

EXTERNAL TESTING SERVICE PROVIDERS

Required when in-house team lacks time, tools or expertise

Fast setup of optimized test environments

Experienced testers with ready-to-use skills

Can fill specific role gaps efficiently

Ideal for specialized or large-scale testing efforts

TEST PLANNING

Testing is a complex process that needs detailed planning on both

- Strategic level – Long-term goals and directions
- Operational level – Execution and day-to-day tasks

Strategic planning is the test manager's responsibility and starts with aligning the project with

- Company's testing policy or strategy
- Project-specific objectives, constraints, risks, and quality requirements

STRATEGIC TEST PLANNING TASKS

Specify the test object(s)

- Define what will be tested (modules, components, systems)
- Identify exclusions

Draft the test objectives

- Clearly define what you're testing for
- Define criteria per test object and for the overall system

Customize the testing process

- Adapt the company process to the project needs
- Define test levels and how they integrate with other project activities

STRATEGIC TEST PLANNING TASKS

Select testing methods and techniques

- Choose suitable testing techniques (manual, automated, etc.)
- Plan necessary training for tools/techniques
- Evaluate test automation options

Specify required infrastructure

- Identify required environment and tools
- Assess existing resources and what needed to be acquired
- Document tool and infrastructure requirements

STRATEGIC TEST PLANNING TASKS

Define test metrics

- Choose metrics to monitor and control the test process
- Define how and when to react to results

Specify the reporting system

- Define what documents/reports to generate (e.g. test logs, defect reports)
- Assign responsibilities for reporting and documentation
- Provide templates and tool configurations

STRATEGIC TEST PLANNING TASKS

Plan costing and testing effort

- Estimate total effort and resource needs
- Define and secure the budget
- Revise cost estimates as needed

OUTCOME: THE TESTING STRATEGY

The result of strategic planning is the test strategy

It's documented in the overall test plan

Should include

- Justification of chosen method
- References to alternative approaches in case of changes

ISO 29119 can be used as a structural reference

A strategy may be

- A master test plan (applies broadly)
- A level test plan (applies to one test level)
- A specialized strategy (e.g. for usability or performance)

OUTCOME: THE TESTING STRATEGY

Strategic planning involves research, discussion, and negotiation

Must be updated as the project progresses and:

- Requirements evolve
- Risks are reassessed

New insights are gained from executed tests or pilot feedback

Some measures may be reduced or dropped based on risk analysis

Both strategic and operational test planning are continuous

Test manager must:

- Stay critical and flexible
- Regularly revise plans as the context changes

SELECTING A TESTING STRATEGY

Choosing a strategy is crucial as it affects how effective and efficient testing process is under project constraints and minimizes the project risks

Two core influence:

- Decision making freedom
- Available knowledge about the project/product

SELECTING A TESTING STRATEGY (BASED ON DECISION MAKING FREEDOM)

Preventative Approach

Testers involved from the beginning

Enables proactive design of strategy

Uses design reviews, early test specification, static analysis

It detects issues early which results in lower cost, higher quality

Essential for safety-critical systems

SELECTING A TESTING STRATEGY (BASED ON DECISION MAKING FREEDOM)

Reactive Approach

Testers involved late in the project

Strategy must adapt to current state

Uses exploratory testing, on-the-fly design and execution

Higher defect density and limited prevention risk

SELECTING A TESTING STRATEGY (BASED ON DECISION MAKING FREEDOM)

“Start early, Stay involved”

A preventative strategy is always recommended

Cost-benefit analysis proves

- Early testing → cost savings
- Late testing → costly fixes

SELECTING A TESTING STRATEGY (BASED ON EXPERIENCE)

Analytical Approach

Data driven

Relies on metrics, models, and systematic analysis

Best when accurate data is available

SELECTING A TESTING STRATEGY (BASED ON EXPERIENCE)

Heuristic Approach

Experience based

Uses expert judgements, rules of thumb

Used when:

- Data is unavailable
- Modeling is too complex
- Time/resources are limited

CONCRETE TESTING STRATEGIES

Real-world strategies often mix analytical, heuristic, preventive, and reactive elements

Common combinations:

- Risk based + Exploratory
- Cost-driven + Automated reuse

CONCRETE TESTING STRATEGIES

Cost based testing

- Maximize cost efficiency
- Prefer simple, broad test instead of in-depth, expensive test cases unless justified
- Suitable for projects with tight budget and timelines

Risk based testing

- Focus on high risk areas
- Analyze project/product risks
- Test where implicit or likelihood of failure is highest
- Suitable for mission-critical systems

CONCRETE TESTING STRATEGIES

Model based testing

- Use models to guide test creation
- Use state machines, fault distribution models, or usage frequency as input
- Suitable for complex systems requiring systematic coverage

Standard or process oriented approach

- Meet external/internal compliance
- Use industry standards (e.g. ISO, IEEE), company policies, legal regulations
- Suitable for regulated industries (e.g. finance, healthcare, aviation)

CONCRETE TESTING STRATEGIES

Directive (consultative) approach

- Rely on expert judgement and stakeholder input
- Based on advice, instincts or experience of domain experts
- Often guided by external stakeholders or consultants
- Suitable for new technologies, unknown domains or startups

Methodical approach

- Apply structured, predefined test conditions
- Use list of common defects, quality attributes, UX/UI or brand standards
- Suitable for UX-driven, QA-mandated projects

CONCRETE TESTING STRATEGIES

Reuse based approach

- Leverage existing assets
- Use test, scripts, environment from previous projects
- Setup testing quickly and programmatically
- Suitable for iterative development or product family testing

Checklist based approach

- Use prior knowledge to guide testing
- Refer to defect lists, know failure modes, risk logs and quality priorities
- Suitable for organizations with a strong testing history

CONCRETE TESTING STRATEGIES

Regression averse approach

- Protect existing functionality
- Use automated regression suites, old test cases and data
- Emphasize test repetition
- Suitable for large, evolving codebases with frequent updates

RISK AND TESTING

Risk helps prioritize test objectives, techniques, and test cases

$$\text{Risk} = (\text{Expected Loss}) \times (\text{Probability of occurrence})$$

Loss – Consequence and cost of product failure

Probability – Likelihood of failure based on product usage

In risk estimation, precise definition of loss and probability is rare; use estimated values or ranges

Risk types are classified into

- Project risks
- Product risks

PROJECT RISKS

Risks that threaten the success or continuation of a project

Categories

1. Supplier issues

Contract breaches, delays, legal issues

2. Organizational issues

Resource shortage, Training failure, Poor communication, Political conflict

3. Process weakness

Inefficient defect/configuration management

PROJECT RISKS

4. Technical issues

Poor/late requirement changes

Unskilled use of new technologies

Defective interim deliverables

Testing environment delays

PRODUCT RISKS

Risks associated with quality and functionality of the deliverable product

Examples:

Software doesn't work or fulfill its purpose

Missing or faulty features

Poor performance or usability

Architecture lacks scalability

Data quality issues (e.g. from migration)

Product causes physical harm or safety issues

RISK MANAGEMENT

Systematic approach by project management to reduce or eliminate risk

Steps:

1. Risk Analysis – Identify potential risks
2. Prioritization – Rank based on impact and likelihood
3. Planning – Define countermeasures
4. Contingency Planning – Prepare for high impact risks

TESTING AS A RISK REDUCER

Testing provides insights into real-world issues

Reduces uncertainty and helps discover hidden risks

Helps confirm that corrective actions are effective

Note: Testing cannot prevent project risks (e.g. sick leave) but can mitigate product related consequences

RISK BASED TESTING

A proactive testing strategy that combats product risks early

Risk-based approach includes:

- Choosing techniques, levels, and types of tests based on risk
- Determining extent of testing (test cases, variants)
- Prioritizing test cases based on risk analysis

Risk evaluation must consider end-user perspective and usage scenarios, also align with insights from stakeholders

Train inexperienced team members to reduce risk

Test high risk components first; Identify major issues early to avoid delays

Avoid equal testing of critical and non-critical features:

- Prevents resource waste
- Ensure adequate focus on important areas

TESTING EFFORT AND COST

Understand how cost impacts test planning and strategy

Testing cost is driven by effort, objectives, and associated risks

Outcome of this process is budget estimation and alignment with test goals

Testing cost should reflect:

- Test objectives
- Component risk level

For low risk components, extent of testing (number of test cases) can be reduced

For high risk components, more rigorous and costly testing is required

KEY FACTORS INFLUENCING TESTING EFFORT

- Development process maturity
- Change frequency and complexity
- Testing process maturity
- Time pressure
- Company stability
- Software quality and testability
- Documentation quality
- System size and complexity
- Application domain complexity

ADDITIONAL INFLUENCING FACTORS

Test infrastructure

Testing tools and environment

Standard processes and techniques

Team and co-workers

Experience with tools, domains, products

Team collaboration and communication

Quality objectives

Regulatory compliance

Testing strategy

Test levels and techniques, scheduling and iterations

ESTIMATING TESTING EFFORT

Many testing techniques are mutually interdependent, hard to quantify factors

Small projects → estimate as a whole; large projects → estimate per level/iteration

Rule of thumb: Testing effort = 25-50% of total development effort

Estimation techniques:

- **Expert based estimation**

Task by task breakdown by experienced testers

Tends to be overly optimistic

- **Metrics based estimation**

Based on previous similar projects or key performance metrics

More realistic and data-driven

COST OF TESTING VS COST OF DEFECT

If testing is reduced:

- More undetected defects
- Higher product risk
- Increased total cost per release

If testing is increased a lot:

- Risk of higher cost or budget overrun

Rule of thumb: Use risk based testing to put resources on high risk components and reduce testing effort on low risk components

Problem: Reducing testing effort can ship product with the undiscovered defects, which can lead towards indirect cost

COST OF FLAWED PRODUCT

Direct cost

Financial losses due to incorrect results

Hardware damage, personal injury

Indirect cost

Customer dissatisfaction

Legal penalties

Brand/image damage

DEFECT CORRECTION COST

Defect correction requires

- Analysis and fixing time
- Regression testing
- Delays in releases
- Retraining users

For risk analysis, estimate cost probability and impact

Risk depends on product type, industry, number/severity of defects, number of installations

Defects in early phases causes cascading effects

Late detection requires rework across phases and results in slower delivery

TEST MANAGEMENT CYCLE

1. Define the testing strategy
2. Plan test execution
3. Initiate and monitor test execution
4. Evaluate and report test results



TEST EXECUTION PLANNING

Convert high-level strategy into actionable test cases

Results in a test execution schedule

Must consider current software status, test results, and available resources

Key influencing factors:

Development stage – If features are missing or changed, test plan need updates

Previous test results – New defects results in new confirmation tests

Resources – Limited staff/time/tools may require cutting or prioritizing tests

TEST EXECUTION PLANNING

Planning the test sequence:

Test execution planning also involves establishing or optimizing the sequence in which the planned tests are to be executed

While sequencing test cases, there must be a balance between test design, test automation, initial execution, regression testing

Run newly created system tests manually first before automating them

TEST EXECUTION PLANNING

Test prioritization:

When resources are constrained, focus on high-priority test cases

Priority criteria:

Usage frequency – More frequent -> higher chance of failure

Risk level – high risk defects or failures with customer impact

User visibility – Issues easily noticed by user should be tested

Requirement importance – Core features > optional features

Non-functional qualities – performance, security etc.

System architecture – critical or failure prone modules

Complexity – complex code is more error prone

Fix Effect – Components needing major rework should be tested early

TEST EXECUTION PLANNING

Entry criteria: “Definition of Ready”

Requirements available

Environment, tools, test data and test object are ready

Starting too early increases cost and risk

Exit criteria: “Definition of Done”

All planned tests executed

Required coverage (test/requirement/code) achieved

Target product quality met

Acceptable residual risk threshold not exceeded

CASE STUDY: EXIT CRITERIA FOR VSR-II SYSTEM TESTING

The VSR-II test cases are assigned one of three priorities:

Priority	What it means
1	Test case must be executed
2	Test case should be executed
3	Test case can be executed

Based on these priorities, the test plan lists the following test case-based exit criteria for system testing VSR-II:

1. All Priority 1 test cases have run without failure
2. At least 60% of Priority 2 test cases have run without failure

Once these exit criteria are fulfilled, test manager decides whether to deliver the test object.

TEST CONTROL

Corrective actions taken to ensure test activities proceed as planned

Supports implementation of test plan and test execution schedule

Initiate planned activities:

- Assign test tasks to team members
- Ensure timely start and completion of these tasks

Respond to changes or issues:

- Quickly reallocate resources (e.g. staff, tools) to address test backlogs
- Improvise solutions when necessary, without waiting for next test cycle

TEST CONTROL

Evaluate corrective measures:

- Check if initiated fixes are effective (e.g. whether an unmet exit criteria has now been met)

Manage deviations:

- Address discrepancies from test plan promptly
- Plan extra test cycles if critical issues arise – though this may delay release and time to market

Communicate clearly:

- Report issues beyond the test team's control to relevant stakeholders
- If test cases are dropped (e.g. due to time constraint), document increased risk and include it in test summary report
- Ensure project leaders are informed of any risk-arising issues promptly and clearly

TEST CYCLE MONITORING

Involves gathering data and feedback related to all testing activities to track progress and ensure objectives are being met

Determine whether testing is progressing as planned

Evaluate if exit criteria are fulfilled

Collected information may include product risk coverage, requirement fulfillment, acceptance criteria completion

Data collection can be done manually or automatically

Metrics defined in the test plan are used to:

- Monitor testing progress
- Check alignment with the test execution schedule and budget
- Evaluate the effectiveness of test techniques and strategies

It enables the test manager to make informed decisions and apply corrective actions if necessary

TEST REPORTS

To summarize and communicate the current state of testing to project managers and other stakeholders

Reports are typically created:

- At the end of an iteration/test cycle
- After completion of a specific testing activity
- Interim reports can be issued in response to unexpected issues

TEST REPORTS

Types of Report:

1. Test Progress Report

- Focus on the current state of testing in the latest iteration
- Brief and iteration specific
- Includes test results, progress updates

2. Test Summary Report

- Covers all completed test activities across iterations leading up to a release
- Used as a basis for product release decisions
- Includes a test manager's evaluation of whether product can be released

DEFECT MANAGEMENT

The process of recording, communicating, managing defects, faults, and issues found during testing

Report creation – starts when a failure or defect is found (e.g. during testing or user report)

Defect logging – each report is structured with consistent information

Assignment – responsibility is defined (e.g. who handles it and when)

Tracking and resolution – monitored until the defect is fixed or closed

Everyone (developer, tester, designer, etc.) must follow the defined process

Create a defect report when a new failure is detected, user reports an issue, a developer finds a fault

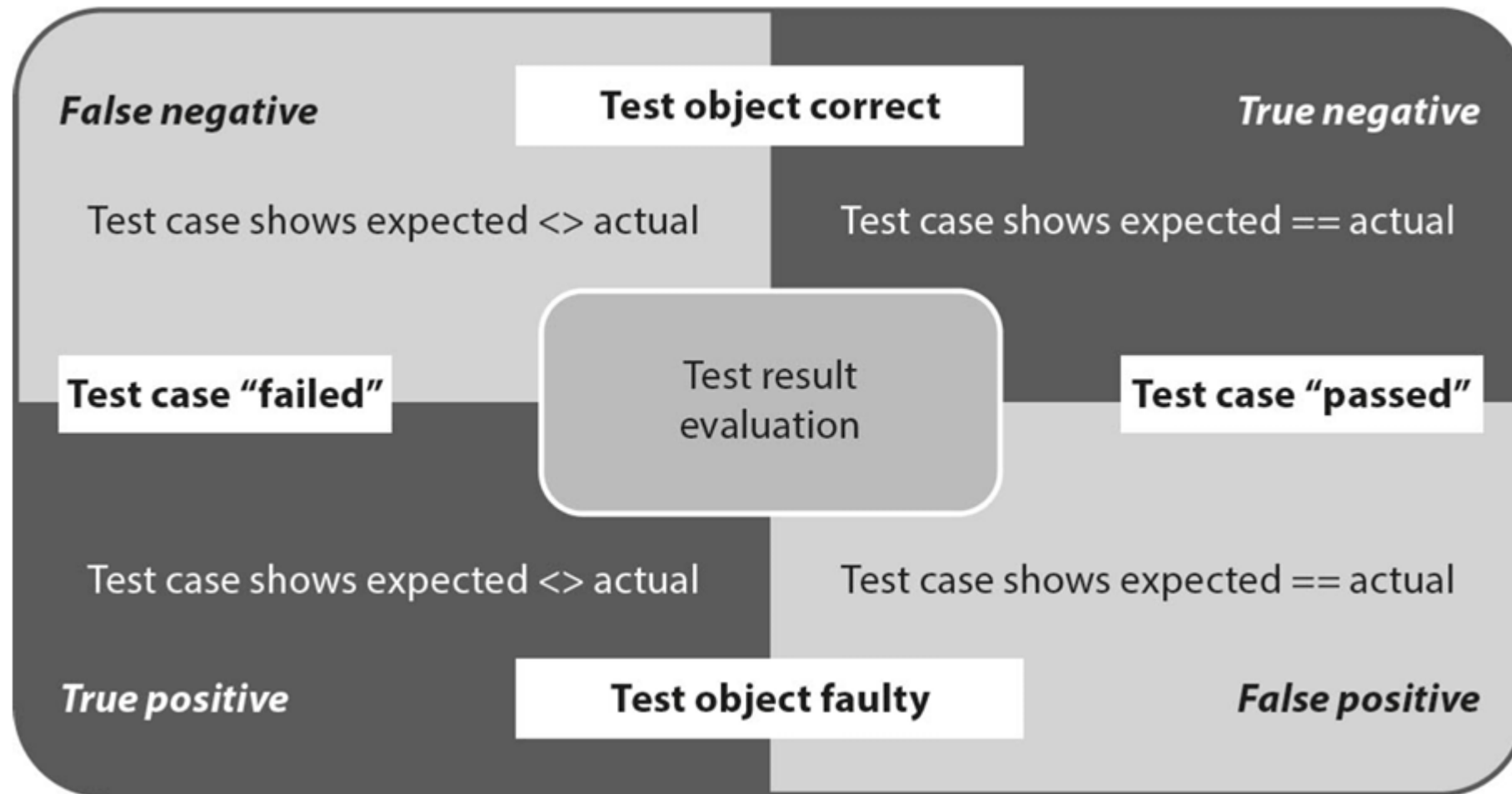
DEFECT MANAGEMENT

After each test run, generate a test log that records:

1. Test case/step
2. Expected behavior
3. Actual behavior
4. Any deviation between the two

Helps to evaluate whether a defect exists

DEFECT MANAGEMENT



DEFECT MANAGEMENT

Classification	Description
True positive	Fault exists; test correctly fails (defect found)
False negative	Fault exist; test doesn't catch it (missed defect)
False positive	No fault; test fails wrongly (e.g. bad test case)
True negative	No fault; test passes as expected

True positive – create or update the defect report

False positive – identify the actual source (e.g. test script, environment)

Duplicate defects – update the existing report with new insights

DEFECT REPORT

Central part of the defect management process

Logged in a defect database

Covers issues like functional defects, requirement errors, documentation faults

Must be communicated clearly among team members

Do not try to analyze or solve the defect – that's the developer's job to which defect report will be forwarded

Report should focus on what happened (effect), where it happened (context), how it affects the systems/user

Defect report must be brief, precise and reproducible (developer should be able to recreate the issue quickly)

DEFECT REPORT

Use a project-wide standard *template*

Helps with consistency across other reports

VSR-II reused a proven template from their earlier project which saved a lot of time defining the format for new project

Template should be adapted to:

- Project size and complexity
- Development life cycle model
- Team size and structure
- Criticality of the system

DEFECT REPORT TEMPLATE

Attribute	What it means
ID	Unique, sequential number
Title	Title and a short summary of the issue in question
Date	Date and (if necessary) the time of the initial report
Author	The person or organization issuing the report
References	
Test object	Name of the test object / test item / configuration item
Version	The exact version of the test object
Platform	Identifiers for the hardware and software and/or test environment in which the problem occurs
Test case	Reference to the test case that revealed the problem
Requirement	Reference to the (customer) requirement that is infringed or not fulfilled because of the defect
Cross references	Cross references to other related reports

DEFECT REPORT

Use defect management tools which helps in:

- Auto-generating defect numbers
- Recording authors automatically
- Enforcing the workflow
- Allows role based access

Documentation should all necessary information required for:

- Fast reproduction of the issue
- Easy localization of faults
- Tracking process status and metrics

CLASSIFYING DEFECT

Helps manage defect correction efficiently

Focuses attention on what most affects the users

Not all issues are equal

- System crash → critical
- UI misalignment → minor

Severity “How bad is it?”

Measures impact on product use

Example:

Class 1 – critical failure

Class 2 – Major impact

Class 3 – Minor inconvenience

CLASSIFYING DEFECT

Priority “How urgent is it?”

Determines order of correction

Based on:

- Blocking other tests
- Fix effort/time
- Release deadlines

Tester ensures that:

- Fixes truly resolves the problem
- No regression or new issues created

CLASSIFYING DEFECT

Status tracking requires end-to-end monitoring of defect analysis and correction process, from discovery and registration to the finished solution

Monitoring is tracked using “status” attribute and rules which determine which changes of status are possible and/or permissible

Different status from which a defect passes according to *defect status model*:

New – Defects are found by tester and provided title and description in defect report

Open – Test manager assigned the defect to the relevant developer

Rejected – Test manager rejected the defect because it was favoring change in requirements

Analysis – Developer identifies causes of defects, estimation correction effort, and so on

Observation – Developer identify exact part where changes are requires

CLASSIFYING DEFECT

Correction – Developer performs the correction; new defects can also result after change

Test – Developer passes the corrected defect to tester for again testing (confirmation testing)

Done – If the confirmation testing performed by tester verified the test (expected = actual)

Flop – If the confirmation testing performed by tester wasn't successful; further analysis is required

Only testers can set status to “done”

If fix causes new issues → create new defect report

Decision can be made by individuals (small teams) or by committees (large teams)

Change control board (CCB) approves or rejects defect fixes

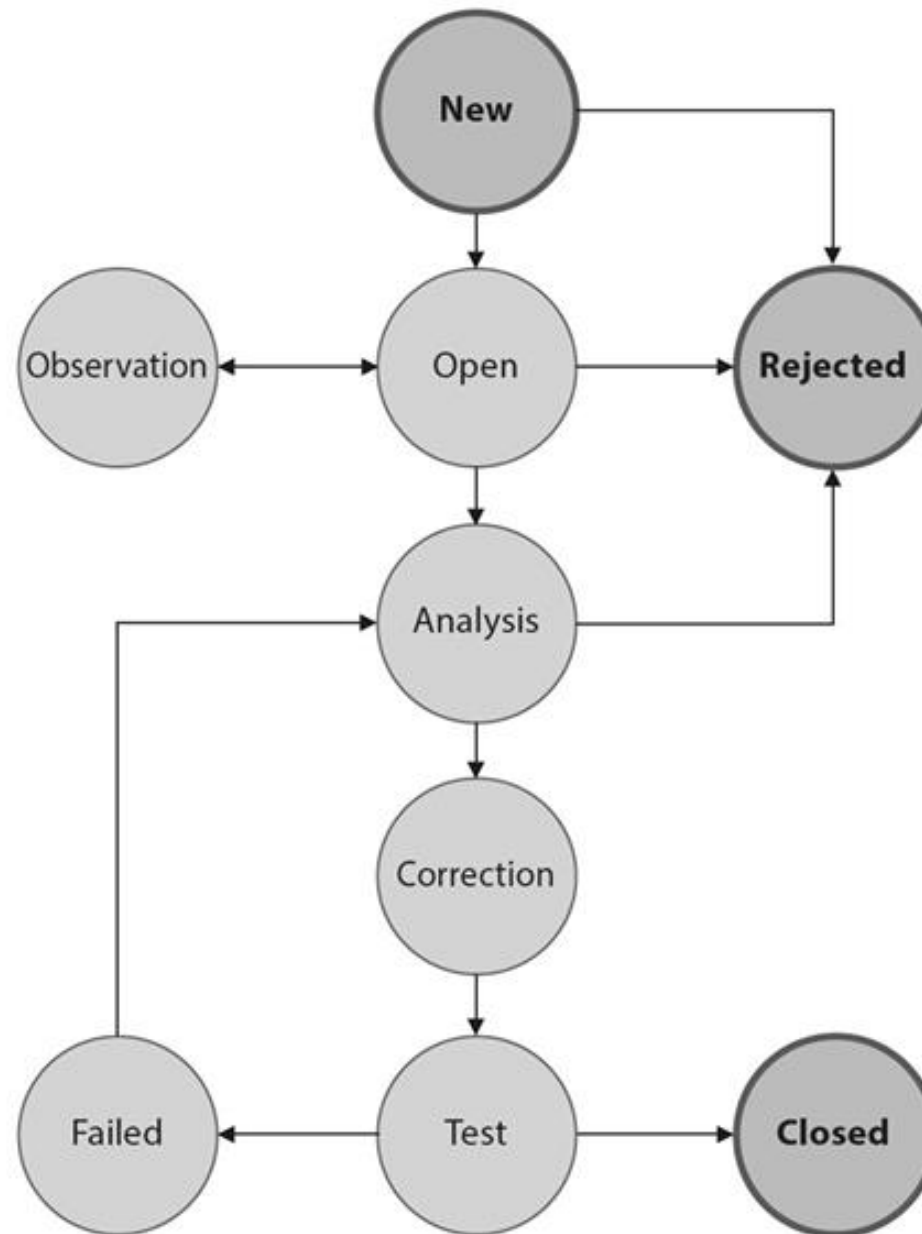


Fig: Defect status model

FAULT CORRECTION PRIORITY

Priority	What it means
1 – Patch	The problem has to be addressed immediately (if necessary temporarily) using a patch
2 – Next release	The correction will be implemented in the next regular release or the next (internal) test object version
3 – Non-urgent	The correction will take place when the affected system components are due for revision anyway
4 – Open	Correction not yet planned

EVALUATION AND REPORTING

Defect database is used for quality assessment, adapting test plans, release decisions

Key metrics are number of defects, correction status and trends (defect rate is flattening)

Defect trends reveal area needing more tests, weak test coverage, requirement issues

As defect's trend analysis helps improving existing processes, VSR-II system has extended its exit criteria by adding further three more criteria:

- All class 1 defects are “done”
- All “patch” priority defects are “done”
- Number of “new” defects per week is stagnating or decreasing

CONFIGURATION MANAGEMENT

To ensure all components of a software system work together correctly, especially in large projects where multiple teams are involved and components exist in different versions

Problem caused by poor CM:

- Developers overwrite each other's work due to unmanaged file access
- Integration difficulties from unclear component version compatibility
- Defect correction and regression testing become harder due to lack of version traceability
- Ambiguity between test results due to unclear test object versions

CONFIGURATION MANAGEMENT

Requirements for effect CM in testing:

Treat test objects, test cases, and test data as configuration item

Maintain version control with change documentation

Link defect reports to affected configuration items

Conduct *configuration audits* to validate completeness and correctness of configurations

CM ensures both development/testing and user environment use valid, consistent configuration

CM plans should be project-specific

[IEEE 828] is a standard for configuration management planning

STANDARDS AND NORMS

To ensure software development and testing adhere to recognized quality and regulatory requirements

Sources of standards:

- **Company standards** – Internal guidelines (e.g. quality manuals, coding rules)
- **Best practices** – Proven, non-standardized techniques
- **Quality management standards** – [ISO 9000] focusing on process requirements
- **Industry specific standards** – [EN 60601] for medical devices
- **Software testing standards** – [ISO 29119] most relevant, outlines professional software testing processes

CONCLUSION

Development and testing activities should remain separate with regard to staff and organization. The clearer this distinction, the more effective testing will be

Testing tasks and roles need to be assigned to people with appropriate specialist skills. As well as technical skills, testers should have comprehensive soft skills too

As well as overall strategic test planning, a test manager's tasks also include operational planning, monitoring, and control for each individual test cycles

The test manager describes the testing strategy (objectives, tasks, tools, and so on) in the test plan

A testing strategy has to strike the best possible compromise between the cost of testing, the available resources, and the potential costs of insufficient testing

CONCLUSION

Tests are prioritized to ensure that less important tests are discarded in the case of a lack of appropriate testing resources

“Risk” is one of the best criteria to use to prioritize tests. Risk-based testing uses information about identified risks to plan and control all aspects of the testing process

Measurable exit criteria provide an objective way to judge when testing is complete. A lack of clear exit criteria increases the risk of testing being terminated randomly

The defect and configuration management processes form the bedrock of an efficient testing process

Defect reports have to be recorded using a unified, project-wide template and must be tracked through all stages of the subsequent analysis and correction processes